

Insertion Sort

Save the next key, shift larger prefix values one position right, and insert the key into the gap.

This complete offline chapter connects ideas, diagrams, exact traces, commented Java and exam practice. Switch themes for reading; printing always uses the light presentation and includes every required trace and answer.

GUIDE 2026-10-07.1 · STANDALONE HTML AND PDF SHARE THE SAME CONTENT

1. The array we will sort

Arrays use zero-based positions. Sorting changes which values occupy those positions. The target is ascending order; record tags would move with their values.



Initial input [4,2,5,1,3]. Five fixed index positions 0..4.

We will distinguish comparisons, swaps, shifts, temporary writes and copy-back writes. They are different units.

2. Definition and plain language

Insertion sort builds a sorted prefix one value at a time. It inserts each next value into the correct location among the earlier sorted values.

Save the next key



Shift larger prefix values



Insert into the gap

The algorithm has a distinct decision and movement rule; each completed pass grows its known-sorted region.

Keep the left-hand cards ordered. Pick up the next card and hold it safely. Move larger cards right until there is a gap after a card no larger than the one you hold.

Hold key

2

4

2

5

1

3

0

1

2

3

4

↓ shift 4 right and insert 2

2

4

5

1

3

0

1

2

3

4

A concrete first pass: highlighted positions explain the choice; the green region records the completed result.

The prefix is already sorted, so comparisons move backwards. Once a prefix value is no larger than the key, every earlier prefix value is also no larger.

3. Vocabulary and the central invariant

Term	Meaning in this chapter
Key	The saved value being inserted, not a permanent array index.
Shift	Copy one prefix value to the next cell; it temporarily leaves duplicate cells.
Gap	The cell at $j + 1$ where the held key will be placed.
i	Original index of the next key.
j	Backwards-moving index within the sorted prefix.

Invariant: Before outer iteration i , prefix $0..i-1$ is sorted. The key remains saved while larger prefix values shift; after insertion prefix $0..i$ is sorted.

4. Step through the complete execution

Predict the next decision before advancing. Next updates both the scene and the visibly marked trace row. Print includes the completed states and full table.

One comparison or write at a time

▼ Full trace table - select a row to jump to its scene

Step	Operation	Reason	Shared array
1	Pass 1, save key 2	key = arr[1] = 2. Save it before shifting overwrites its cell.	[4, 2, 5, 1, 3]
2	Pass 1, shift 4 right	arr[1] = arr[0]. Temporary duplicate 4 is safe because key 2 is held separately.	[4, 4, 5, 1, 3]
3	Pass 1, insert into index 0	arr[0] = key = 2. Prefix 0..1 is now sorted.	[2, 4, 5, 1, 3]
4	Pass 2, save key 5	key = arr[2] = 5. Save it before shifting overwrites its cell.	[2, 4, 5, 1, 3]
5	Pass 2, stop at index 1	4 > 5 → false. The gap is index 2.	[2, 4, 5, 1, 3]
6	Pass 2, insert into index 2	arr[2] = key = 5. Prefix 0..2 is now sorted.	[2, 4, 5, 1, 3]
7	Pass 3, save key 1	key = arr[3] = 1. Save it before shifting overwrites its cell.	[2, 4, 5, 1, 3]
8	Pass 3, shift 5 right	arr[3] = arr[2]. Temporary duplicate 5 is safe because key 1 is held separately.	[2, 4, 5, 5, 3]
9	Pass 3, shift 4 right	arr[2] = arr[1]. Temporary duplicate 4 is safe because key 1 is held separately.	[2, 4, 4, 5, 3]
10	Pass 3, shift 2 right	arr[1] = arr[0]. Temporary duplicate 2 is safe because key 1 is held separately.	[2, 2, 4, 5, 3]
11	Pass 3, insert into index 0	arr[0] = key = 1. Prefix 0..3 is now sorted.	[1, 2, 4, 5, 3]
12	Pass 4, save key 3	key = arr[4] = 3. Save it before shifting overwrites its cell.	[1, 2, 4, 5, 3]
13	Pass 4, shift 5 right	arr[4] = arr[3]. Temporary duplicate 5 is safe because key 3 is held separately.	[1, 2, 4, 5, 5]
14	Pass 4, shift 4 right	arr[3] = arr[2]. Temporary duplicate 4 is safe because key 3 is held separately.	[1, 2, 4, 4, 5]
15	Pass 4, stop at index 1	2 > 3 → false. The gap is index 2.	[1, 2, 4, 4, 5]
16	Pass 4, insert into index 2	arr[2] = key = 3. Prefix 0..4 is now sorted.	[1, 2, 3, 4, 5]

Pass 1, insert into index 0



Indexes remain fixed. Highlighted cells are the current comparison or write; green prefix/suffix is known sorted.

arr[0] = key = 2. Prefix 0..1 is now sorted.

Pass 2, insert into index 2



Indexes remain fixed. Highlighted cells are the current comparison or write; green prefix/suffix is known sorted.

arr[2] = key = 5. Prefix 0..2 is now sorted.

Pass 3, insert into index 0



Indexes remain fixed. Highlighted cells are the current comparison or write; green prefix/suffix is known sorted.

arr[0] = key = 1. Prefix 0..3 is now sorted.

Pass 4, insert into index 2



Indexes remain fixed. Highlighted cells are the current comparison or write; green prefix/suffix is known sorted.

arr[2] = key = 3. Prefix 0..4 is now sorted.

Final result: [1, 2, 3, 4, 5].

5. Exam-style completed-pass / return table

One row per outer-loop pass. These are the completed states to reproduce when an exam asks for the array after each pass.

Pass	Array before	Evaluated comparisons / decisions	Array after	Key comparisons	Shifts
1	[4,2,5,1,3]	4 > 2 → true	[2,4,5,1,3]	1	1
2	[2,4,5,1,3]	4 > 5 → false	[2,4,5,1,3]	1	0
3	[2,4,5,1,3]	5 > 1 → true; 4 > 1 → true; 2 > 1 → true	[1,2,4,5,3]	3	3

Pass	Array before	Evaluated comparisons / decisions	Array after	Key comparisons	Shifts
4	[1,2,4,5,3]	5 > 3 → true; 4 > 3 → true; 2 > 3 → false	[1,2,3,4,5]	3	2

6. What actually moves, and why

Pass 3 saves key 1 before shifting [2, 4, 5]. The temporary duplicates in the array are expected; the held key is the missing value.

Held key

1

2

4

5

5

3

0

1

2

3

4

5 copied

↓ shift 4, then 2

1

2

4

5

3

0

1

2

3

4

key inserted

A shift writes one array cell; the final insertion is another array write. Six shifts plus four insertions gives ten array writes on the main input.

7. Fully commented Java implementation

JAVA

```
public static void insertionSort(int[] arr) {
    // Prefix 0..i-1 is already sorted before each insertion.
    for (int i = 1; i < arr.length; i++) {
        int key = arr[i]; // Preserve this value while its cell is overwritten.
        int j = i - 1;    // Compare backwards through the sorted prefix.
        // Check j first: && avoids evaluating arr[-1].
        // Strict > keeps equal values in their original order (stability).
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j]; // SHIFT: copy one value one cell right.
            j--;
        }
        // The gap is after the last value <= key, or at index 0.
        arr[j + 1] = key;
    }
}
```

Write it from scratch

- Start at the second value.
- Save the key.
- Walk backwards through the sorted prefix.
- Shift strictly larger values right.
- Insert into $j + 1$.

8. Recursion, calls and returns

JAVA

```
public static void insertionSortRec(int[] arr, int n) {  
    // A prefix of zero or one value is already sorted.  
    if (n <= 1) return;  
    // Suspend this frame until the smaller prefix has been sorted.  
    insertionSortRec(arr, n - 1);  
    // Work happens ON RETURN: insert the final value into the sorted prefix.  
    int key = arr[n - 1], j = n - 2;  
    while (j >= 0 && arr[j] > key) {  
        arr[j + 1] = arr[j];  
        j--;  
    }  
    arr[j + 1] = key;  
    // First call: insertionSortRec(arr, arr.length).  
}
```

CALL STACK - NEWEST FRAME AT THE TOP

insertionSortRec(n=1) Base case returns immediately
insertionSortRec(n=2) Waiting: Insert 2 after child returns
insertionSortRec(n=3) Waiting: Insert 5 after child returns
insertionSortRec(n=4) Waiting: Insert 1 after child returns
insertionSortRec(n=5) Waiting: Insert 3 after child returns

Calls descend to $n=1$; insertions execute as $n=2,3,4,5$ return. The arrays after each return match the pass table.

9. Complete runnable program

Save as `InsertionDemo.java`. The code includes the method(s) and a main entry point.

JAVA

```
import java.util.Arrays;

public class InsertionDemo {
    public static void insertionSort(int[] arr) {
        // Prefix 0..i-1 is already sorted before each insertion.
        for (int i = 1; i < arr.length; i++) {
            int key = arr[i]; // Preserve this value while its cell is overwritten.
            int j = i - 1;    // Compare backwards through the sorted prefix.
            // Check j first: && avoids evaluating arr[-1].
            // Strict > keeps equal values in their original order (stability).
            while (j >= 0 && arr[j] > key) {
                arr[j + 1] = arr[j]; // SHIFT: copy one value one cell right.
                j--;
            }
            // The gap is after the last value <= key, or at index 0.
            arr[j + 1] = key;
        }
    }

    public static void main(String[] args) {
        int[] values = {4, 2, 5, 1, 3};
        insertionSort(values);
        System.out.println(Arrays.toString(values));
    }
}
```

EXPECTED CONSOLE OUTPUT

[1, 2, 3, 4, 5]

10. Descending order: change the decision

Change `arr[j] > key` to `arr[j] < key`. Shift smaller prefix values right, stopping after a value at least as large as the key.

Keep the bounds and movement rules consistent. Reversing a final array is a different procedure and does not reproduce the descending trace of this implementation.

11. Five worked examples

Each case has input and result diagrams, evaluated decisions or merge inputs, exact counts, and a final state.

1. Nearly sorted

1

2

4

3

5

0

1

2

3

4

↓ ascending insertion sort

1

2

3

4

5

0

1

2

3

4

The trace below connects the input diagram to the final result.

Pass	Before	Evaluated conditions	After	Key comparisons
1	[1,2,4,3,5]	$1 > 2 \rightarrow \text{false}$	[1,2,4,3,5]	1
2	[1,2,4,3,5]	$2 > 4 \rightarrow \text{false}$	[1,2,4,3,5]	1
3	[1,2,4,3,5]	$4 > 3 \rightarrow \text{true}; 2 > 3 \rightarrow \text{false}$	[1,2,3,4,5]	2
4	[1,2,3,4,5]	$4 > 5 \rightarrow \text{false}$	[1,2,3,4,5]	1

5 key comparisons; 1 shifts.

2. Reverse order

5

4

3

2

1

0

1

2

3

4

↓ ascending insertion sort

1

2

3

4

5

0

1

2

3

4

The trace below connects the input diagram to the final result.

Pass	Before	Evaluated conditions	After	Key comparisons
1	[5,4,3,2,1]	$5 > 4 \rightarrow \text{true}$	[4,5,3,2,1]	1

Pass	Before	Evaluated conditions	After	Key comparisons
2	[4,5,3,2,1]	$5 > 3 \rightarrow \text{true}; 4 > 3 \rightarrow \text{true}$	[3,4,5,2,1]	2
3	[3,4,5,2,1]	$5 > 2 \rightarrow \text{true}; 4 > 2 \rightarrow \text{true}; 3 > 2 \rightarrow \text{true}$	[2,3,4,5,1]	3
4	[2,3,4,5,1]	$5 > 1 \rightarrow \text{true}; 4 > 1 \rightarrow \text{true}; 3 > 1 \rightarrow \text{true}; 2 > 1 \rightarrow \text{true}$	[1,2,3,4,5]	4

10 key comparisons; 10 shifts.

3. Duplicate values

2

2

1

3

0

1

2

3

↓ ascending insertion sort

1

2

2

3

0

1

2

3

The trace below connects the input diagram to the final result.

Pass	Before	Evaluated conditions	After	Key comparisons
1	[2,2,1,3]	$2 > 2 \rightarrow \text{false}$	[2,2,1,3]	1
2	[2,2,1,3]	$2 > 1 \rightarrow \text{true}; 2 > 1 \rightarrow \text{true}$	[1,2,2,3]	2
3	[1,2,2,3]	$2 > 3 \rightarrow \text{false}$	[1,2,2,3]	1

4 key comparisons; 2 shifts.

4. Uneven length

38

27

43

3

9

0

1

2

3

4

↓ ascending insertion sort

3

9

27

38

43

0

1

2

3

4

The trace below connects the input diagram to the final result.

Pass	Before	Evaluated conditions	After	Key comparisons
1	[38,27,43,3,9]	$38 > 27 \rightarrow \text{true}$	[27,38,43,3,9]	1
2	[27,38,43,3,9]	$38 > 43 \rightarrow \text{false}$	[27,38,43,3,9]	1
3	[27,38,43,3,9]	$43 > 3 \rightarrow \text{true}$; $38 > 3 \rightarrow \text{true}$; $27 > 3 \rightarrow \text{true}$	[3,27,38,43,9]	3
4	[3,27,38,43,9]	$43 > 9 \rightarrow \text{true}$; $38 > 9 \rightarrow \text{true}$; $27 > 9 \rightarrow \text{true}$; $3 > 9 \rightarrow \text{false}$	[3,9,27,38,43]	4

9 key comparisons; 7 shifts.

5. Descending target

8

3

5

1

0

1

2

3

↓ descending insertion sort

8

5

3

1

0

1

2

3

The trace below connects the input diagram to the final result.

Pass	Before	Evaluated conditions	After	Key comparisons
1	[8,3,5,1]	$8 < 3 \rightarrow \text{false}$	[8,3,5,1]	1

Pass	Before	Evaluated conditions	After	Key comparisons
2	[8,3,5,1]	$3 < 5 \rightarrow \text{true}; 8 < 5 \rightarrow \text{false}$	[8,5,3,1]	2
3	[8,5,3,1]	$3 < 1 \rightarrow \text{false}$	[8,5,3,1]	1

4 key comparisons; 1 shifts. Change $\text{arr}[j] > \text{key}$ to $\text{arr}[j] < \text{key}$. Shift smaller prefix values right, stopping after a value at least as large as the key.

12. Properties and why the complexity occurs

Property	Result	Reason
Best	$O(n)$	Already sorted input makes one failed key comparison per insertion.
Average / worst	$O(n^2)$	Reverse input shifts across every earlier prefix cell.
Auxiliary space	$O(1)$ iterative; $O(n)$ recursive	One saved key versus suspended recursive frames.
Stable / in place / adaptive	Yes / yes / yes	Strict $>$ never shifts an equal value past the key; few inversions mean few shifts.

Pass 1		$n - 1$ comparisons
Pass 2		$n - 2$ comparisons
Last pass		1 comparison

Worst-case triangular work: $(n - 1) + (n - 2) + \dots + 1 = n(n - 1)/2$.

On the main input there are 8 key comparisons, 6 shifts and 4 insertion writes. The guard $j \geq 0$ is not a key comparison. Reverse input gives the triangular worst-case number of shifts; sorted input makes $n-1$ failed key comparisons.

13. When to choose it

A strong choice for small or nearly sorted arrays. It is simple, stable, and adaptive; it can also serve as the small-input stage of a larger sorting algorithm.

14. Common mistakes, symptoms and repairs

Attempt	What goes wrong	Repair
Overwrite <code>arr[i]</code> without saving it	The key is lost during shifting.	Save <code>int key = arr[i]</code> first.
Evaluate <code>arr[j]</code> before <code>j >= 0</code>	May read <code>arr[-1]</code> at the left edge.	Place the bounds condition first with <code>&&</code> .
Insert at <code>arr[j]</code>	Writes before the identified gap.	Write <code>arr[j + 1] = key</code> .
Describe every shift as a swap	Counts and intermediate states become wrong.	A shift performs one array write; a swap exchanges two values.

15. Exam-style practice and worked answers

A. Trace [8, 3, 5, 1] in descending order. Show every completed pass/merge. [8]

Pass	Before	After	Key comparisons
1	[8,3,5,1]	[8,3,5,1]	1
2	[8,3,5,1]	[8,5,3,1]	2
3	[8,5,3,1]	[8,5,3,1]	1

Final result [8,5,3,1]. Change `arr[j] > key` to `arr[j] < key`. Shift smaller prefix values right, stopping after a value at least as large as the key.

B. Explain stability with tagged equal keys. [4]

Strict inversion comparisons do not move a value past an equal one. Using `>=` (or `<=` descending) would remove that guarantee.

C. Give the time and space, then justify both. [5]

Best: $O(n)$. Already sorted input makes one failed key comparison per insertion. Average / worst: $O(n^2)$. Reverse input shifts across every earlier prefix cell. Auxiliary space: $O(1)$ iterative; $O(n)$ recursive. One saved key versus suspended recursive frames. Stable / in place / adaptive: Yes / yes / yes. Strict `>` never shifts an equal value past the key; few inversions mean few shifts.

D. Debug the code before tracing it. [4]

Mistake	Symptom	Fix
Overwrite <code>arr[i]</code> without saving it	The key is lost during shifting.	Save <code>int key = arr[i]</code> first.
Evaluate <code>arr[j]</code> before <code>j >= 0</code>	May read <code>arr[-1]</code> at the left edge.	Place the bounds condition first with <code>&&</code> .

E. What happens for `[]` and `[7]`? [2]

Both are already sorted. No key comparisons or value movements are required. Merge sort returns at its base case; the iterative methods never enter a pass.

16. Test-ready summary and mastery check

Save the next key, shift larger prefix values one position right, and insert the key into the gap.

- I can introduce every variable before using it.
- I can draw the current array with index labels.
- I can evaluate conditions with the actual values.
- I can distinguish pass summaries from individual operations.
- I can trace descending and duplicate cases.
- I can justify stability, time and auxiliary space.
- I can test empty, singleton, sorted and reverse input.

CSI247 visual study guide · Insertion Sort · 2026-10-07.1

Structural reference: the author-supplied CSI247 guide, with topic-specific adaptation and repo-owned teaching content. All traces in this artifact are generated by deterministic simulations of its stated procedures. Counts distinguish key comparisons from loop tests, probes, swaps, shifts and array writes.